# CS683

# Assignment 2

**Ishaan Gupta - 24B1081**
**Tushar Yadav - 24B1257**
**Rohan Shukla - 24B1274**
**Parth Chafle - 24B3920**

# Contents


**1 Setup and methodology** **2**

**2 Task 1A: IP-stride prefetcher** **3**

**3 Task 1B: Stride Pattern Prefetcher (SPP)** **4**
- 3.1 Design . . . 4
- 3.2 Results . . . 5
- 3.3 Storage overhead . . . 7
- 3.4 Accuracy thresholds: what and how . . . 7
- 3.5 Why these degrees . . . 8

**4 Task 2A: Instruction-conscious RRIP (I-RRIP)** **9**
- 4.1 Exposing the instruction bit . . . 9
- 4.2 Policy and rationale . . . 9
- 4.3 Results . . . 11

**5 Task 2B: Dynamic I-RRIP** **13**
- 5.1 Leader sets . . . 13
- 5.2 Decision logic . . . 14
- 5.3 Threshold sensitivity (5 values) . . . 15
- 5.4 Results . . . 16

**6 Task 3: SPP with prefetch-aware dynamic I-RRIP** **19**
- 6.1 Handling of prefetch packets: rationale . . . 19
- 6.2 Results . . . 21

**A Tuning experiments** **24**
- A.1 Task 1B: SPP (L2C: LRU) . . . 24
- A.2 Task 2A: I-RRIP (no prefetcher) . . . 26
- A.3 Task 2B: Dynamic I-RRIP (no prefetcher) . . . 27
- A.4 Task 3: SPP + prefetch-aware dynamic I-RRIP . . . 28

# Chapter 1

# Setup and methodology

**Simulation.** ChampSim (PA2 repository), single core, 25M warmup and 25M simulated instructions on each of the three provided traces (`Trace1`, `Trace2`, `Trace3`). The L2C is 1024 sets × 8 ways (512 KB).

**Configurations.**

| Name | L1D prefetcher | L2C replacement |
|---|---|---|
| `baseline` | none | LRU |
| `ip_stride` | IP-stride (Task 1A) | LRU |
| `spp` | SPP (Task 1B) | LRU |
| `rrip` | none | SRRIP (reference for Task 2) |
| `irrip` | none | I-RRIP (Task 2A) |
| `dyn_irrip` | none | Dynamic I-RRIP (Task 2B) |
| `spp_dyn_irrip` | SPP | Dynamic I-RRIP, prefetch awareness off (ablation) |
| `spp_pf_dyn_irrip` | SPP | Prefetch-aware dynamic I-RRIP (Task 3) |

**Metrics.** As defined in the problem statement: Speedup = $\text{IPC}/\text{IPC}_{\text{baseline}}$, Coverage = $(\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses})/\text{L1D load misses}_{\text{baseline}}$, accuracy and MPKI from the ChampSim statistics. The "mean" column of every table and plot is the geometric mean for IPC and speedup and the arithmetic mean otherwise. In the speedup plots the baseline is the dotted line at 1.

**Parameter tuning.** Every parameter of the three policies is a compile-time knob. We tuned them one group at a time around a reasoned default (Task 1 with LRU at L2, Task 2 with no prefetcher, Task 3 with both fixed), one step on each side of the default, and kept a new value only if it improved geometric-mean speedup by more than 0.1% (`scripts/tune.py`, all runs in `results/tuning/`). The values chosen this way are marked `TUNED` in the code. Only the experiments that answer a deliverable are reported below.

# Chapter 2

# Task 1A: IP-stride prefetcher

`prefetcher/ip_stride.l1d_pref` keeps a table of 24 trackers, fully associative with LRU replacement, indexed by the low 8 bits of the load IP. Each tracker stores the IP, the last cache-line address it accessed and the last stride.

On every L1D access:

1. Look up the IP. On a miss, allocate the LRU tracker, record the line address and return (no stride is known yet).
2. Compute the signed stride between the current and the last line address. A zero stride is ignored.
3. If the stride equals the stored stride (seen twice in a row), issue prefetches for $A + k \cdot$ stride, $k = 1..3$ (degree 3), stopping at the 4 KB page boundary. Each prefetch fills the L1D if fewer than half of the L1D MSHRs are busy, and only the L2 otherwise.
4. Update the last address and stride and the LRU order.

Its limits motivate Task 1B. It only recognises a constant stride, so any IP with a repeating multi-stride pattern never prefetches; the degree is fixed whatever the accuracy; and 24 trackers cover few IPs. On our traces it reaches only 2.8% mean coverage at 35.6% accuracy (Table 3.1).

# Chapter 3

# Task 1B: Stride Pattern Prefetcher (SPP)

## 3.1 Design

Code: `prefetcher/spp.l1d_pref`. Three tables and a few throttle registers.

- **Signature Table (ST)**, one entry per IP (256 entries, fully associative, LRU): last line address, last stride, a stride signature, a 2-bit FSM state and a uniform-stride confidence.
- **Pattern Table (PT)**, 1024 entries indexed by the signature: the stride that followed this signature, with a 2-bit confidence. It is shared by all IPs.
- **Prefetch filter**, 256 entries, direct mapped on the line address. It drops duplicate prefetches and is the accuracy checker: a demand access that hits an unused filter entry counts as a useful prefetch.

**Uniform vs. non-uniform.** The per-IP FSM is INIT $\rightarrow$ UNIFORM on the first stride, and UNIFORM $\rightarrow$ NON-UNIFORM as soon as a stride differs from the previous one. It goes back to UNIFORM only after 3 equal strides in a row; a single repeat is not enough, because a pattern such as $(+1, +2, +2, +2, +3)^n$ contains short runs of equal strides. In UNIFORM state with a confirmed stride the prefetcher issues $A + k \cdot s$, so uniform strides have priority. Otherwise it walks the PT.

**Signature.** sig $\leftarrow ((\text{sig} \ll 3) \oplus \text{stride}) \bmod 2^{10}$, which keeps about three strides of history. Three is the minimum the spec's example needs: after $+2, +2$ the next stride can be $+2$ or $+3$, and only the stride before them ($+1$ or $+2$) tells the two apart. The signature and the PT are trained on every access, in both states, so the PT already knows the pattern when an IP falls back to it.

**Degree $>$ 1 (lookahead).** From the IP's signature the prefetcher reads the predicted stride, adds it to the address, updates a speculative signature with it and repeats, up to the current degree. It stops when the PT confidence is too low or the next address leaves the 4

KB page. Strides are 7-bit signed (±63 lines, the reach of a page); larger jumps are stored as a "far" code so they still enter the history.

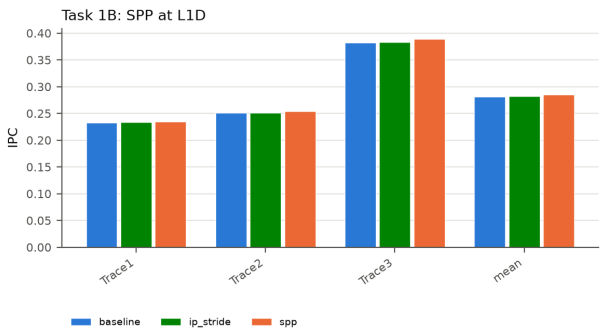
**Accuracy-based throttling.** Every 1024 demand loads, the accuracy useful/issued is computed from two counters that are halved after each decision (so recent behaviour dominates) and mapped to a degree through Table 3.3. Degree 0 is a shadow mode: predictions are written to the filter but not issued, so accuracy is still measured and the prefetcher can turn itself back on.

## 3.2 Results

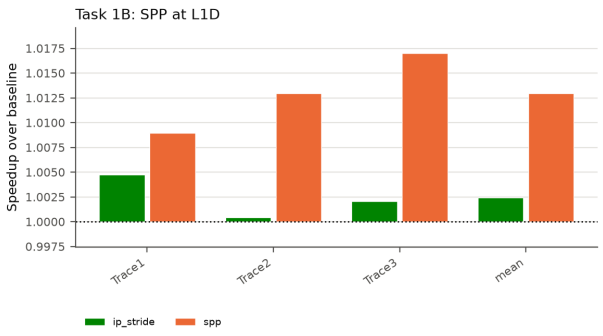
| | IPC | | | Speedup | | | L1D load MPKI | | |
|---|---|---|---|---|---|---|---|---|---|
| Trace | base | ip_str | spp | base | ip_str | spp | base | ip_str | spp |
| Trace1 | 0.2323 | 0.2335 | 0.2344 | 1 | 1.0048 | 1.0090 | 10.68 | 10.12 | 9.52 |
| Trace2 | 0.2512 | 0.2513 | 0.2544 | 1 | 1.0004 | 1.0129 | 13.54 | 13.44 | 12.97 |
| Trace3 | 0.3821 | 0.3829 | 0.3886 | 1 | 1.0021 | 1.0170 | 3.40 | 3.32 | 2.86 |
| mean | 0.2815 | 0.2821 | 0.2851 | 1 | 1.0024 | 1.0130 | 9.21 | 8.96 | 8.45 |

| | Accuracy (%) | | Coverage (%) | |
|---|---|---|---|---|
| Trace | ip_stride | spp | ip_stride | spp |
| Trace1 | 52.71 | 57.51 | 5.24 | 10.86 |
| Trace2 | 38.18 | 53.92 | 0.74 | 4.16 |
| Trace3 | 15.99 | 43.14 | 2.51 | 15.91 |
| mean | 35.63 | 51.53 | 2.83 | 10.31 |

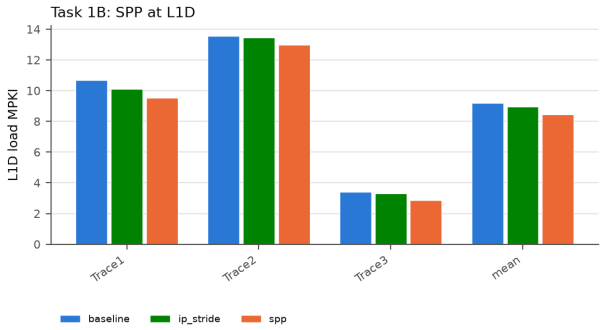
Table 3.1: Task 1B: SPP vs. IP-stride (L2C: LRU).



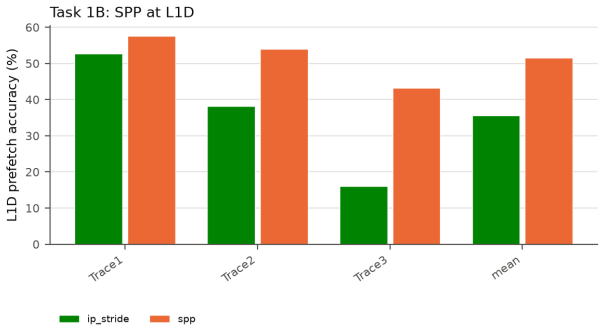

(a) IPC




(b) Speedup




(a) L1D load MPKI




(b) Prefetch accuracy

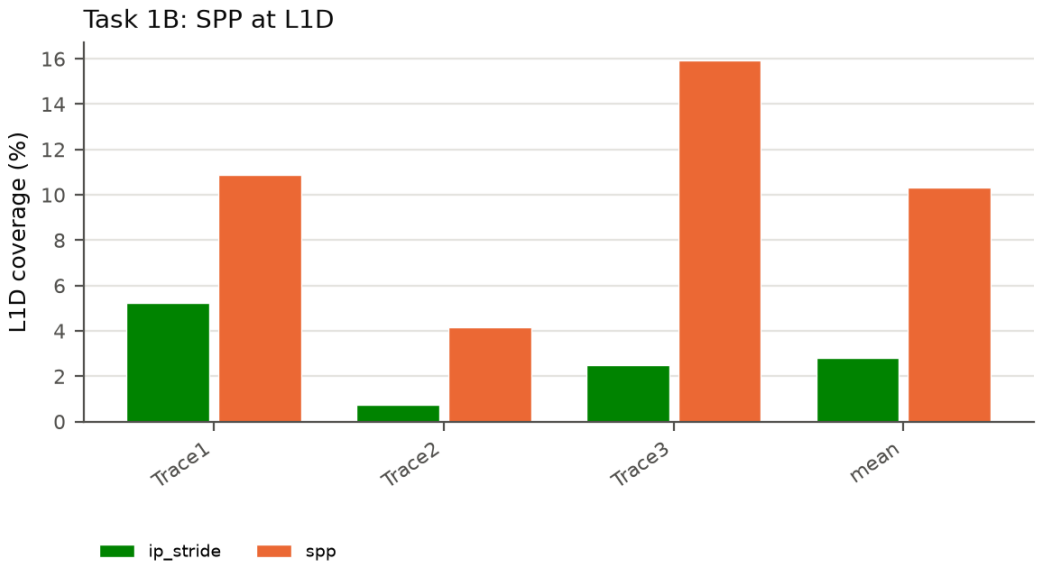



Figure 3.3: Task 1B: L1D coverage.

SPP beats IP-stride on every trace and every metric: 3.6× the coverage at 16 points higher accuracy, for a 1.30% geometric-mean speedup (IP-stride: 0.24%). Most of SPP's prefetches come from the PT, i.e. from non-uniform patterns that IP-stride cannot predict (85,453 of 116,607 issued on Trace1; 67,469 of 74,878 on Trace3). Trace3 shows the difference most: IP-stride keeps prefetching strides that have broken and is 16% accurate, while SPP falls back to the PT and is 43% accurate.

## 3.3 Storage overhead

| Structure | Fields (bits) | Entries | Bits/entry | Total bits |
|---|---|---|---|---|
| Signature Table | valid 1, IP tag 10, last addr 16, last stride 7, signature 10, FSM state 2, uniform conf 2, LRU 8 | 256 | 56 | 14,336 |
| Pattern Table | stride 7, confidence 2 | 1024 | 9 | 9,216 |
| Prefetch filter | valid 1, partial tag 8, useful 1, class 1 | 256 | 11 | 2,816 |
| Throttle regs | issued 16, useful 16, epoch counter 11, degree 3 | 1 | 46 | 46 |
| **Total** (3.2 KB) | | | | **26,414** |

Table 3.2: SPP storage, bit by bit (also printed by every run).

To keep the tables lightweight: only the low 16 bits of the last address are kept (strides are computed mod $2^{16}$, which is exact for every prefetchable stride of $\pm 63$); the IP is hashed to 10 bits; and the filter uses an 8-bit partial tag. An alias can only make the measured accuracy slightly optimistic. The ST size was tuned: with 64 entries (15,534 bits in total), 54% / 70% of the loads of Trace1 / Trace2 found no ST entry for their IP. With 256 entries this drops to 23% / 47%, and speedup rises from 1.0079 to 1.0130 for 10.9 Kbit (32 entries: 1.0052, 128: 1.0112). Every other size (PT 256 or 4096 entries, filter 128 or 512) changed speedup by less than 0.1%, so it was kept at the smaller or default value.

## 3.4 Accuracy thresholds: what and how

| Accuracy range | Degree | Reason |
|---|---|---|
| $\geq 0.75$ | 4 | most predictions right, including lookahead steps |
| 0.50–0.75 | 3 | right more often than wrong; one step shallower |
| 0.25–0.50 | 2 | gain $\approx$ waste; first prediction + one lookahead |
| 0.10–0.25 | 1 | mostly wrong; only the most confident guess |
| $< 0.10$ | 0 | shadow mode (measure, do not issue) |

Table 3.3: Accuracy thresholds and degrees (initial degree 2).

**How the thresholds were decided.** At accuracy $a$, each prefetch saves a miss with probability $a$ and with probability $1 - a$ costs an L1D line it pollutes, a PQ slot and L2 bandwidth.

- Below $a \approx 0.5$ the expected waste equals the expected gain, so the degree must stop growing there.
- Below 0.25, three of four prefetches pollute, so only one guess is allowed.
- 0.75 is where even the deeper lookahead steps (whose errors compound) are still mostly right.

- 0.10 is a "toxic" floor, as in feedback-directed prefetching (Srinath et al., HPCA 2007, uses 0.40/0.75; ours is that scheme one step finer).

The accuracy used for throttling is measured by the filter ("the predicted line was demanded while still tracked"), which measures prediction quality. That is what the degree controls.

**Checked by experiment.** Moving every boundary by about 0.1 in either direction (0.65/0.40/0.20/0.05 and 0.85/0.65/0.35/0.15) changed speedup by at most 0.02%. That is expected: the filter accuracy is 69–70% on all three traces, well inside the 0.50–0.75 band and away from its edges.

## 3.5 Why these degrees

Each extra lookahead step is less accurate than the one before it, so the degree grows with measured accuracy. 4 blocks ahead is enough lead time to hide an L2 hit; more mostly overruns short streams and the page boundary (already 26K page-boundary stops on Trace1). At run time the prefetcher spends most epochs at degree 3, and at degree 4 in regular phases: on Trace1 2,666 and 1,090 of 4,229 epochs, and on Trace3, the most regular trace, about half the epochs at degree 4. Shadow mode was never needed.

Against no throttling (degree 3 always), a more aggressive table (6/4/3/1/0) and a more conservative one (3/2/1/1/0), speedup stayed within 0.01%. The accuracy moved as intended: 42.6% for the aggressive table, 45.4% for ours and 48.0% for the conservative one. We kept the middle table because it gives the same speedup with fewer useless prefetches than the aggressive one, and it adapts if a workload's accuracy drops.

# Chapter 4

# Task 2A: Instruction-conscious RRIP (I-RRIP)

## 4.1 Exposing the instruction bit

`update_replacement_state` gained an `uint8_t isInstruction` argument in `inc/cache.h`, in every replacement policy and at every call site in `src/cache.cc`, which passes the packet's `instruction` field (the BTB in `src/ooo_cpu.cc` passes 0). A line counts as an instruction line for `LOAD`/`PREFETCH` packets with `instruction` set, the same rule ChampSim uses for "L2C instruction load MPKI". At eviction the policy reads the per-line `block.instruction` bit.

## 4.2 Policy and rationale

Code: `replacement/irrip.l2c_repl`. The baseline is SRRIP (2-bit RRPV: insert at 2, hit $\rightarrow$ 0, evict at 3). All three parts were modified.

| | SRRIP | I-RRIP |
|---|---|---|
| Insertion | every fill at 2 | instruction at 1, data at 3, writeback at 2 |
| Promotion | hit $\rightarrow$ 0 | instruction hit $\rightarrow$ 0; data hit $\rightarrow$ RRPV$-1$; writeback hit: none |
| Eviction | first line at 3 | data at 3, else data at 2, else first line at 3 |

**Why prioritise instructions at all.** An instruction miss stalls the front end, so nothing behind it can be fetched, while a data miss is partly hidden by the out-of-order window. These traces have an instruction footprint far larger than the L1I (up to 19 L2 instruction MPKI under LRU), so the L2 is where instructions are served. The policy therefore trades data hits for instruction hits.

**Insertion.**

- *Data at 3*: a data line is the next victim of its set unless it is re-referenced first. Most L2 data lines are streams already filtered by the L1D and have no second use at L2. This is distant (BRRIP-style) insertion, applied to data only.
- *Instructions at 1, not 0*: at 0 a line of straight-line code that runs once would need three ageing rounds to leave, which pins dead code. At 1 it outlives data inserted at the same time by two rounds, enough for loops and hot functions to be re-referenced.
- *Writebacks at 2*: they may be re-read, but they are not demands.

**Promotion.** An instruction hit is proof of reuse of the expensive kind, so it gets full protection at once. A data line climbs one step per hit: inserted at 3, it needs three hits to reach 0, so only data that is reused repeatedly competes with instructions. Hot data still gets there, so it is not starved. A writeback hit does not promote, because a writeback means the L1D is evicting the line, i.e. the line is *not* in use (SRRIP promotes on it).

**Eviction.** Ageing is unchanged. Among the lines at RRPV 3 a data line goes first. If only instruction lines are at 3, a data line at 2 is evicted instead. This overrides the RRPV order by at most one step, so a dead instruction line still leaves once no data line is at 2 or 3: priority without pinning. The tie-break changed the victim 180K–210K times per run on Trace1/Trace2.

**Why this is the best variant we found.** Each part was compared with its alternatives on all traces (geometric-mean speedup):

| Part | Variant | Speedup |
|---|---|---|
| Insertion | data at 2 (SRRIP value) | 1.0081 |
| | **data at 3** | **1.0100** |
| | instruction at 0 | 1.0075 |
| Promotion | **data hit → RRPV−1** | **1.0081** |
| | data hit → 0 (SRRIP) | 1.0075 |
| | data hit → RRPV−2 | 1.0074 |
| Eviction | first line at 3 (SRRIP) | 1.0062 |
| | data before instruction at 3 | 1.0081 |
| | **+ data at 2 before instruction at 3** | **1.0133** |
| Combined | **final I-RRIP** | **1.0135** |

Table 4.1: Task 2A design alternatives (each row changes one part of the untuned I-RRIP: data insertion at 2, tie-break only).

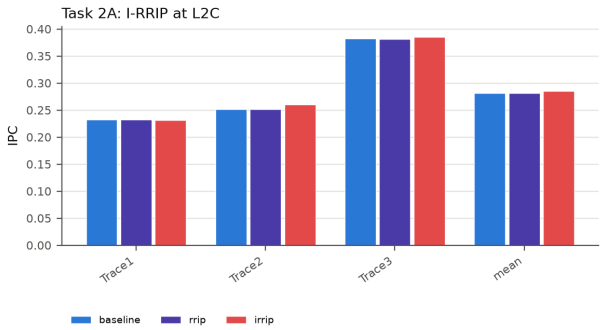
Every change towards SRRIP behaviour lost, and every change towards stronger instruction priority won.

## 4.3 Results

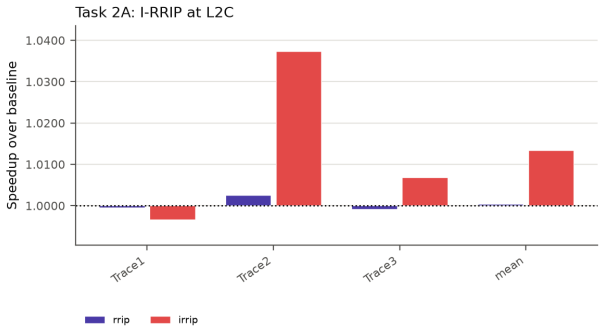
| | IPC | | | Speedup | | |
|---|---|---|---|---|---|---|
| Trace | LRU | SRRIP | I-RRIP | LRU | SRRIP | I-RRIP |
| Trace1 | 0.2323 | 0.2322 | 0.2316 | 1 | 0.9995 | 0.9966 |
| Trace2 | 0.2512 | 0.2518 | 0.2605 | 1 | 1.0026 | 1.0374 |
| Trace3 | 0.3821 | 0.3817 | 0.3847 | 1 | 0.9991 | 1.0068 |
| mean | 0.2815 | 0.2816 | 0.2852 | 1 | 1.0004 | 1.0135 |

| | L2C load MPKI | | | L2C instr. MPKI | | | L2C data MPKI | | |
|---|---|---|---|---|---|---|---|---|---|
| Trace | LRU | SRRIP | I-RRIP | LRU | SRRIP | I-RRIP | LRU | SRRIP | I-RRIP |
| Trace1 | 17.70 | 17.64 | 18.87 | 10.34 | 10.56 | 9.08 | 7.36 | 7.08 | 9.79 |
| Trace2 | 27.36 | 25.75 | 20.29 | 18.85 | 17.78 | 8.86 | 8.50 | 7.96 | 11.44 |
| Trace3 | 3.95 | 4.04 | 3.89 | 1.12 | 1.20 | 0.67 | 2.83 | 2.84 | 3.22 |
| mean | 16.34 | 15.81 | 14.35 | 10.11 | 9.85 | 6.20 | 6.23 | 5.96 | 8.15 |

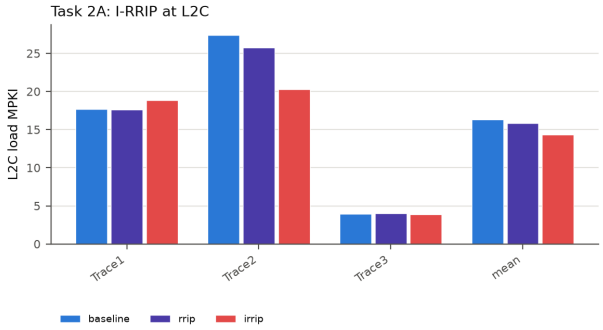
Table 4.2: Task 2A results (no prefetcher).



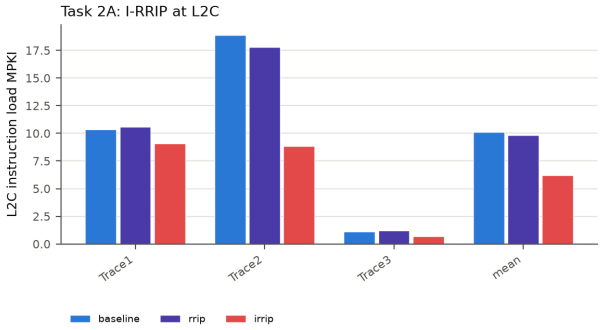

(a) IPC




(b) Speedup




(a) L2C load MPKI




(b) L2C instruction MPKI

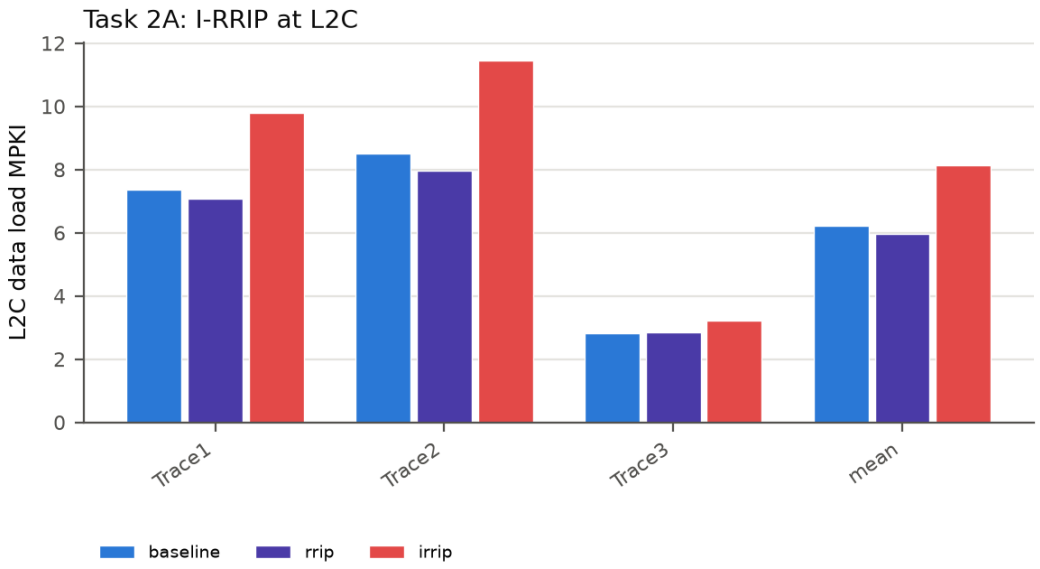



Figure 4.3: Task 2A: L2C data load MPKI.

I-RRIP cuts L2 instruction MPKI by 39% (53% on Trace2) at the cost of 31% more data MPKI. Total L2 MPKI falls 12% and IPC follows the instruction side: +1.35% overall, +3.7% on Trace2. Trace1 loses 0.3%: its extra data misses cost more than the instruction misses it saves. A static policy cannot avoid that, and it is the motivation for Task 2B. SRRIP alone gains almost nothing (+0.04%), so the gain comes from the instruction awareness, not from RRIP.

# Chapter 5

# Task 2B: Dynamic I-RRIP

Code: `replacement/dyn_irrip.l2c_repl`. Follower sets run either SRRIP or I-RRIP, chosen by one global 1-bit selector that is recomputed every 16,384 L2 demand accesses (an epoch). The I-RRIP half of the duel uses the I-RRIP rules as they were when Task 2B was tuned (data inserted at 2, tie-break only).

## 5.1 Leader sets

**16 SRRIP leaders and 16 I-RRIP leaders out of 1024 sets (3.1%).**

- *Placement (complement select, as in DIP).* The sets form 16 regions of 64. In region $r$, set offset $r$ is an SRRIP leader and offset $63 - r$ is an I-RRIP leader. Both kinds are spread over the whole index space and never collide, and the kind is a function of the index bits, so it costs no storage.
- *Why 16.* Leader sets cost performance: they are pinned to one policy, and half of them run the losing one. What they buy is the accuracy of the measured ratio $R$ (below). With 16 SRRIP leaders an epoch gives $\approx 256$ demand samples, and with counter halving about 500 samples are in effect. The standard error of $R$ is then $\sqrt{0.25/500} \approx 0.02$, well below the distance between the threshold (0.40) and the mean ratios of Trace2 (0.60) and Trace3 (0.27). Only Trace1 ($R \approx 0.45$) sits near the threshold, and for Trace1 the two policies are within 0.1% of each other, so a noisy choice there costs nothing.
- *Measured.* With the original decision logic, 16 leaders gave 1.0076, 32 gave 1.0062 and 64 gave 1.0066. With the final logic, 16 and 32 give the same 1.0077, so 16 pins half as many sets at equal decision quality.

## 5.2 Decision logic

Per leader kind, four 16-bit counters count instruction and total demand accesses and misses (`LOAD`, `RFO`, translation; prefetches and writebacks are excluded). At the end of each epoch:

$$R = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}} \quad \text{(measured on the SRRIP leaders)}$$

$$\text{followers} = \begin{cases} \text{I-RRIP} & R > T(1+h) \\ \text{SRRIP} & R < T(1-h) \\ \text{unchanged} & \text{otherwise} \end{cases} \qquad T = 0.40,\ h = 0.05$$

then every counter is halved, so the decision follows program phases.

- $R$ is the spec's formula. Counting a miss also as an access weights instruction lines that are being lost. It is measured on the leaders that always run SRRIP, i.e. the unprotected behaviour; measured on I-RRIP sets, $R$ would fall as soon as I-RRIP worked, and the selector would oscillate.
- The relative hysteresis band ($\pm 5\%$ of $T$) stops the selector from flipping every epoch when $R$ sits on the threshold.
- *Alternative we built and rejected: a duel guard.* Classic set dueling compares misses on the two leader groups, cost $= c \cdot M_{\text{instr}} + M_{\text{data}}$. We implemented it both alone and as a veto on top of the ratio ("ratio AND guard", which was our first default):

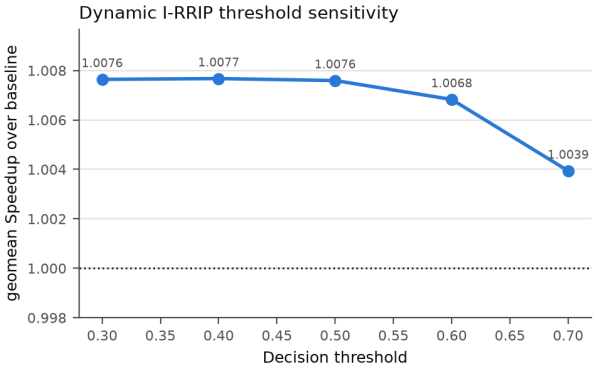
| Decision logic | Speedup |
|---|---|
| ratio AND guard ($c = 2$, $T = 0.25$) | 1.0062 |
| guard only (classic duel) | 1.0063 |
| ratio AND guard, $c = 4$ / $c = 8$ | 1.0065 / 1.0065 |
| ratio only, $T = 0.25$ | 1.0075 |
| **ratio only, $T = 0.40$ (final)** | **1.0077** |

The guard counts misses, and I-RRIP always shows more extra data misses than saved instruction misses. On Trace2 the I-RRIP leaders saved 350 instruction misses and added 2,655 data misses, so the guard vetoed I-RRIP in 69% of Trace2's epochs, although static I-RRIP is faster there (+2.4% at the time, +3.7% after tuning). Miss counts do not capture that an instruction miss stalls the whole front end. The instruction share $R$ does, so the final policy uses the ratio alone.

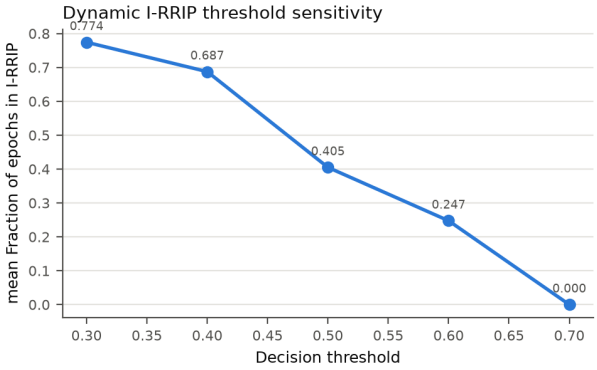
## 5.3 Threshold sensitivity (5 values)

| $T$ | 0.30 | 0.40 | 0.50 | 0.60 | 0.70 |
|---|---|---|---|---|---|
| Speedup Trace1 | 0.9997 | 1.0002 | 1.0007 | 1.0007 | 1.0007 |
| Speedup Trace2 | 1.0233 | 1.0228 | 1.0226 | 1.0200 | 1.0113 |
| Speedup Trace3 | 1.0001 | 1.0001 | 0.9997 | 0.9999 | 0.9999 |
| **Speedup geomean** | 1.0076 | **1.0077** | 1.0076 | 1.0068 | 1.0039 |
| L2C MPKI (mean) | 14.38 | 14.38 | 14.36 | 14.50 | 14.95 |
| L2C instr. MPKI (mean) | 7.77 | 7.82 | 7.98 | 8.19 | 8.97 |
| L2C data MPKI (mean) | 6.61 | 6.56 | 6.38 | 6.30 | 5.98 |
| I-RRIP epochs Trace1 | 1.00 | 0.83 | 0 | 0 | 0 |
| I-RRIP epochs Trace2 | 0.94 | 0.93 | 0.91 | 0.74 | 0 |
| I-RRIP epochs Trace3 | 0.38 | 0.31 | 0.31 | 0 | 0 |

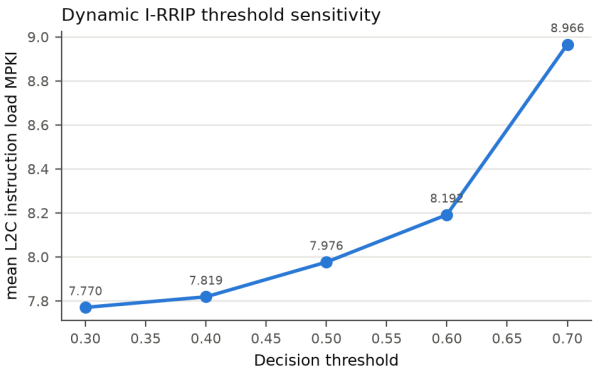
Table 5.1: Sensitivity to the threshold $T$ (mean $R$: Trace1 0.45, Trace2 0.59, Trace3 0.27).



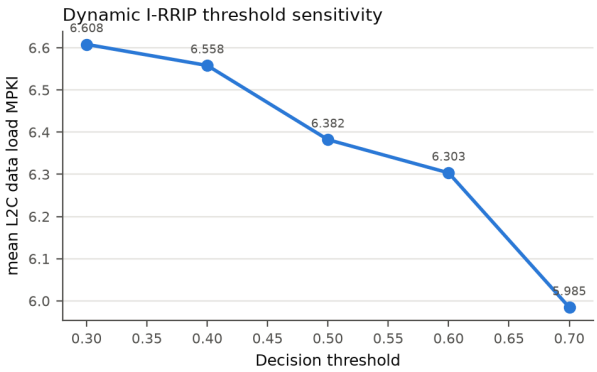

(a) Geomean speedup vs. $T$




(b) Fraction of epochs in I-RRIP vs. $T$




(a) L2C instruction MPKI vs. $T$




(b) L2C data MPKI vs. $T$

**What the study shows.**

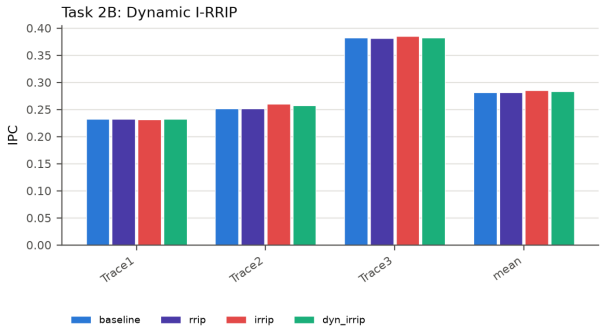
- Each trace leaves I-RRIP once $T$ passes its own ratio: Trace1 (mean 0.45) at $T = 0.5$, Trace2 (0.59) between 0.6 and 0.7, and Trace3 (0.27, with a few early high-ratio epochs) at 0.6.
- Instruction MPKI rises and data MPKI falls monotonically with $T$.
- Performance has a broad plateau for $T$ between 0.30 and 0.50, and drops once Trace2, the trace that gains most from I-RRIP, is switched to SRRIP ($-0.38\%$ at 0.70).
- 0.40 is the peak and in the middle of the plateau, so it is the most robust choice. It keeps Trace2 in I-RRIP, and lets Trace1, which does slightly better with SRRIP, leave I-RRIP some of the time.
- An earlier sweep at $T = 0.05$–$0.40$, with the guard, was flat (1.0059–1.0064). Those thresholds lay below the ratios of Trace1 and Trace2, so the guard alone decided. That is why the final sweep is centred on the measured ratios.

## 5.4 Results

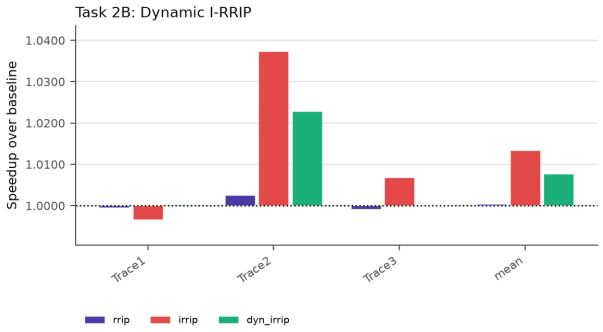
| | IPC | | | | Speedup | | | |
|---|---|---|---|---|---|---|---|---|
| Trace | LRU | SRRIP | I-RRIP | dyn | LRU | SRRIP | I-RRIP | dyn |
| Trace1 | 0.2323 | 0.2322 | 0.2316 | 0.2324 | 1 | 0.9995 | 0.9966 | 1.0002 |
| Trace2 | 0.2512 | 0.2518 | 0.2605 | 0.2569 | 1 | 1.0026 | 1.0374 | 1.0228 |
| Trace3 | 0.3821 | 0.3817 | 0.3847 | 0.3821 | 1 | 0.9991 | 1.0068 | 1.0001 |
| mean | 0.2815 | 0.2816 | 0.2852 | 0.2836 | 1 | 1.0004 | 1.0135 | 1.0077 |

| | L2C load MPKI | | | | L2C instr. MPKI | | | | L2C data MPKI | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Trace | LRU | SR | IR | dyn | LRU | SR | IR | dyn | LRU | SR | IR | dyn |
| Trace1 | 17.70 | 17.64 | 18.87 | 17.56 | 10.34 | 10.56 | 9.08 | 9.82 | 7.36 | 7.08 | 9.79 | 7.74 |
| Trace2 | 27.36 | 25.75 | 20.29 | 21.67 | 18.85 | 17.78 | 8.86 | 12.58 | 8.50 | 7.96 | 11.44 | 9.09 |
| Trace3 | 3.95 | 4.04 | 3.89 | 3.90 | 1.12 | 1.20 | 0.67 | 1.06 | 2.83 | 2.84 | 3.22 | 2.85 |
| mean | 16.34 | 15.81 | 14.35 | 14.38 | 10.11 | 9.85 | 6.20 | 7.82 | 6.23 | 5.96 | 8.15 | 6.56 |

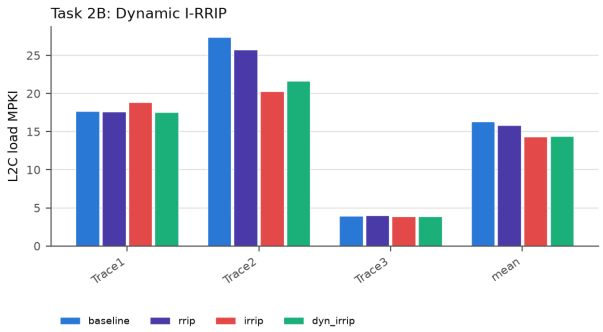
Table 5.2: Task 2B results (no prefetcher; I-RRIP = tuned Task 2A policy). Fraction of epochs in I-RRIP: 0.83 / 0.93 / 0.31.



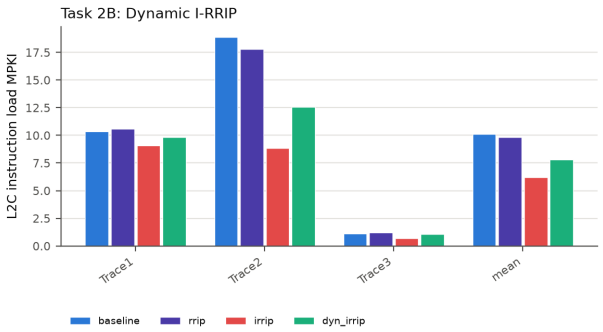

(a) IPC



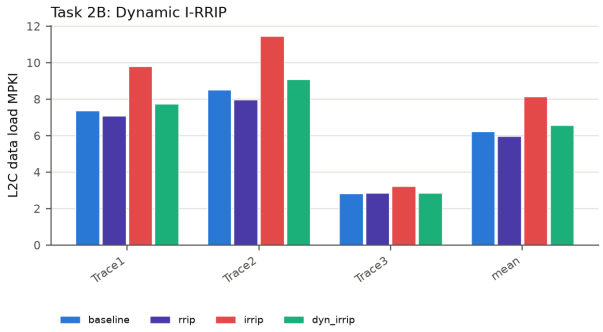

(b) Speedup



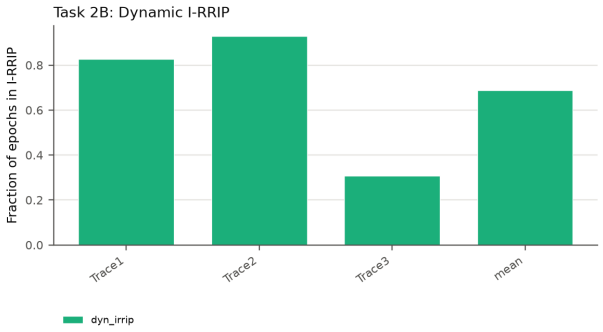

(a) L2C load MPKI




(b) L2C instruction MPKI




(a) L2C data MPKI




(b) Fraction of epochs in I-RRIP

Dynamic I-RRIP does what it was designed for. On Trace1, where static I-RRIP loses 0.3%, it is the best policy (1.0002), with the lowest total L2 MPKI of all four. On Trace2 it stays in I-RRIP 93% of the time. It has the lowest data MPKI of the instruction-aware policies, and almost the same total L2 MPKI as static I-RRIP (14.38 vs. 14.35). Its geometric-mean speedup (1.0077) is below the tuned static I-RRIP (1.0135) for two reasons. Its I-RRIP half uses the untuned Task 2A rules. And on these three traces I-RRIP is the better policy in most phases, so every epoch spent in SRRIP costs a little. Set dueling pays off when phases disagree, as in Trace1.

Storage: 2-bit RRPV and an instruction bit per line (24,576 bits), plus $2 \times 4 \times 16$ counter bits, a 15-bit epoch counter and the 1-bit selector: 24,720 bits in total.

# Chapter 6

# Task 3: SPP with prefetch-aware dynamic I-RRIP

## 6.1 Handling of prefetch packets: rationale

Code: the `PREFETCH_AWARE` paths of `replacement/dyn_irrip.l2c_repl`. With SPP at the L1D, the L2 also receives `PREFETCH` packets. 96–99% of SPP's prefetches are `FILL_L1`: the L1D gets the copy and the core's later reads hit there, so the L2 copy is rarely read. Treated as demand data they would be inserted next to instruction lines, promoted when an L1D prefetch hits them, and counted in the duel. The policy handles them as follows; it applies in every set, leaders included, so the duel stays fair.

| Event | Handling | Reason |
|---|---|---|
| Prefetch fill, `FILL_L1` | RRPV 3 (first to go) | The L1D holds the useful copy; the L2 copy is insurance (PACMan, Wu et al. MICRO 2011). |
| Prefetch fill, `FILL_L2` | RRPV 2 (as demand data) | L1D MSHRs were busy, so the L2 copy is the only one the core will read. |
| Prefetch fill, high L2 prefetch reuse | RRPV 2 | If prefetched lines are being re-read from L2 ($\geq$ 50%), distant insertion would be wrong. |
| Prefetch request hits in L2 | no promotion | An L1D prefetch is not a reuse by the core. |
| First demand hit on a prefetched line | RRPV $\leftarrow$ demand insertion value | This access is the miss the prefetch saved, not evidence of reuse; later hits promote normally. |
| Victim selection | unused prefetched line first, then data, then instructions | Protects the instruction lines dynamic I-RRIP keeps from SPP's traffic. |
| Duel counters | prefetches never counted | They would dilute $R$, which is meant to measure demand instruction pressure. |

No new bus bits are needed. The packet type gives `PREFETCH`, the fill level comes from the MSHR entry being filled, and the per-line prefetch bit already in `BLOCK` marks lines that were prefetched and not yet demanded. Extra state is two 16-bit reuse counters and a flag, 8,225 bits counting the prefetch bit, for 32,945 bits in total.

The measurements confirm the premise. On Trace1, 54,469 prefetch fills went in at RRPV 3, 54,160 prefetched lines left the L2 never demanded, and only 2,078 were demanded. L2 prefetch reuse is 3.7% / 2.4% / 1.6% on the three traces, so the high-reuse rule never fired; it is a safeguard for workloads whose L1D thrashes.

## 6.2 Results

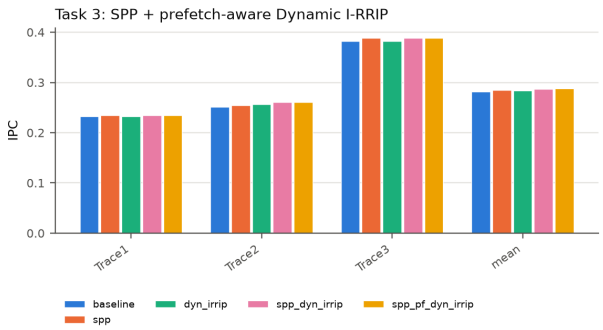
| | IPC | | | | | Speedup | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Trace | base | spp | dyn | spp+dyn | **T3** | base | spp | dyn | spp+dyn | **T3** |
| Trace1 | 0.2323 | 0.2344 | 0.2324 | 0.2345 | **0.2345** | 1 | 1.0090 | 1.0002 | 1.0093 | **1.0094** |
| Trace2 | 0.2512 | 0.2544 | 0.2569 | 0.2605 | **0.2606** | 1 | 1.0129 | 1.0228 | 1.0373 | **1.0377** |
| Trace3 | 0.3821 | 0.3886 | 0.3821 | 0.3886 | **0.3887** | 1 | 1.0170 | 1.0001 | 1.0170 | **1.0174** |
| mean | 0.2815 | 0.2851 | 0.2836 | 0.2874 | **0.2875** | 1 | 1.0130 | 1.0077 | 1.0211 | **1.0214** |

| | Accuracy (%) | | | Coverage (%) | | |
|---|---|---|---|---|---|---|
| Trace | spp | spp+dyn | T3 | spp | spp+dyn | T3 |
| Trace1 | 57.51 | 57.54 | 57.47 | 10.86 | 10.87 | 10.84 |
| Trace2 | 53.92 | 54.07 | 53.87 | 4.16 | 4.14 | 4.13 |
| Trace3 | 43.14 | 42.77 | 43.14 | 15.91 | 15.82 | 15.96 |
| mean | 51.53 | 51.46 | 51.49 | 10.31 | 10.28 | 10.31 |

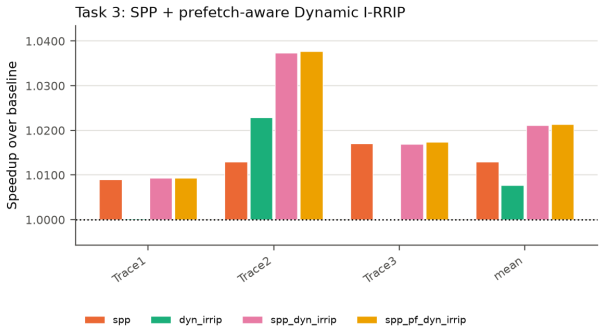
| | L1D load MPKI | | | | L2C load MPKI | | | |
|---|---|---|---|---|---|---|---|---|
| Trace | spp | dyn | spp+dyn | T3 | spp | dyn | spp+dyn | T3 |
| Trace1 | 9.52 | 10.68 | 9.52 | 9.52 | 16.79 | 17.56 | 16.59 | 16.55 |
| Trace2 | 12.97 | 13.54 | 12.98 | 12.98 | 26.92 | 21.67 | 21.06 | 20.95 |
| Trace3 | 2.86 | 3.40 | 2.87 | 2.86 | 3.28 | 3.90 | 3.23 | 3.19 |
| mean | 8.45 | 9.21 | 8.45 | 8.45 | 15.66 | 14.38 | 13.63 | 13.56 |

| | L2C instr. MPKI | | | | L2C data MPKI | | | |
|---|---|---|---|---|---|---|---|---|
| Trace | spp | dyn | spp+dyn | T3 | spp | dyn | spp+dyn | T3 |
| Trace1 | 10.39 | 9.82 | 9.84 | 9.80 | 6.39 | 7.74 | 6.75 | 6.76 |
| Trace2 | 18.95 | 12.58 | 12.50 | 12.40 | 7.97 | 9.09 | 8.56 | 8.55 |
| Trace3 | 1.15 | 1.06 | 1.08 | 1.05 | 2.14 | 2.85 | 2.15 | 2.14 |
| mean | 10.16 | 7.82 | 7.81 | 7.75 | 5.50 | 6.56 | 5.82 | 5.81 |

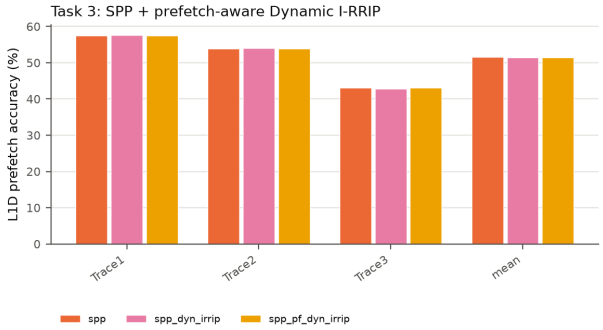
Table 6.1: Task 3. spp = SPP + LRU; dyn = dynamic I-RRIP alone; spp+dyn = SPP + dynamic I-RRIP without prefetch awareness (ablation); T3 = `spp_pf_dyn_irrip`. Baseline L1D MPKI 10.68 / 13.54 / 3.40, L2C MPKI 17.70 / 27.36 / 3.95.



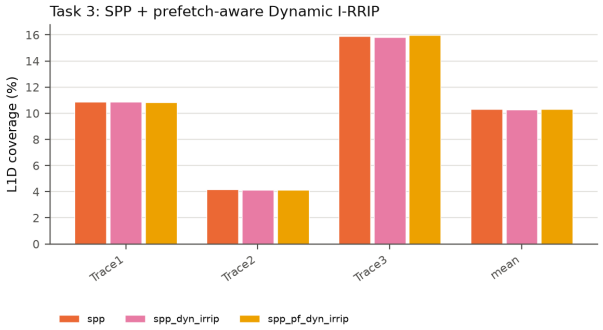

(a) IPC



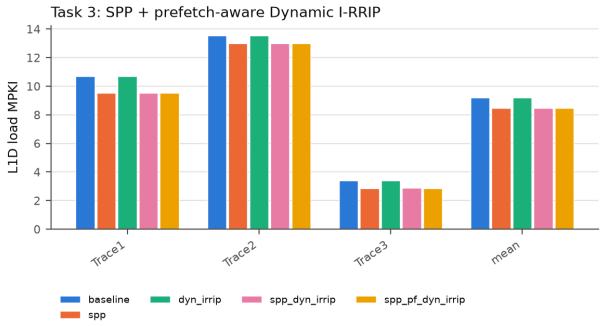

(b) Speedup



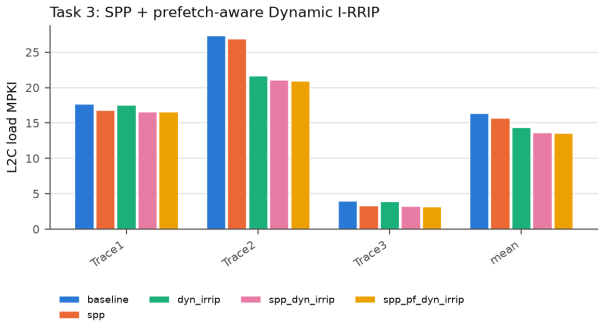

(a) L1D prefetch accuracy



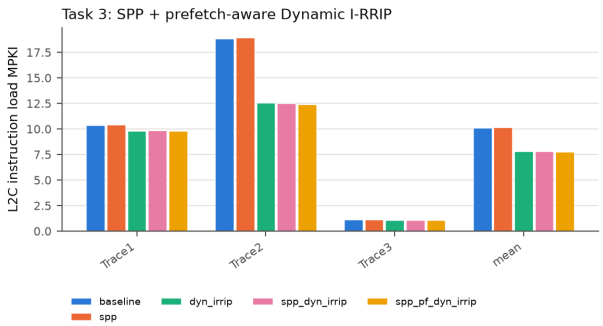

(b) L1D coverage



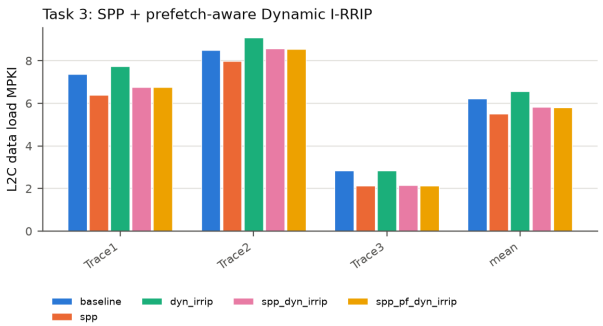

(a) L1D load MPKI




(b) L2C load MPKI




(a) L2C instruction MPKI




(b) L2C data MPKI

**Task 3 outperforms both standalone SPP and dynamic I-RRIP in IPC on every trace** (geomean 0.2875 vs. 0.2851 and 0.2836; speedup 1.0214 vs. 1.0130 and 1.0077).

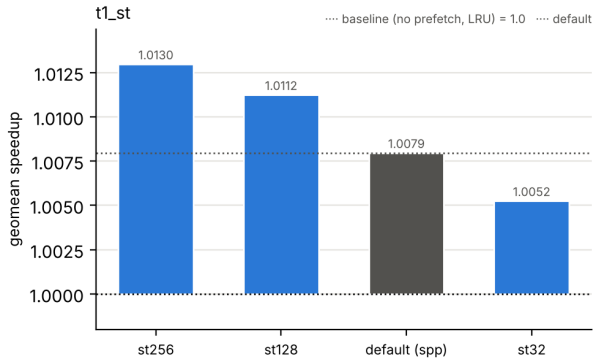
- The two components remove different misses: SPP removes L1D data misses (10% coverage) and dynamic I-RRIP removes L2 instruction misses (−23%). Their gains are therefore nearly additive ($1.30\% + 0.77\% = 2.07\%$ vs. 2.11% for the plain pair).
- Prefetch awareness adds a further 0.03%, entirely on the L2 side: L2 instruction MPKI 7.81 → 7.75 and total 13.63 → 13.56, with L1D accuracy and coverage unchanged. The gain is small because SPP's prefetches are a small share of L2 fills (61K–117K per 25M instructions), so there was little pollution to remove.
- Turning each mechanism off on its own (distant insertion, first use as insertion, unused-prefetch-first eviction) or enabling promotion on prefetch hits cost 0.01–0.04% each (tuning runs). Every piece points the right way.

# Appendix A

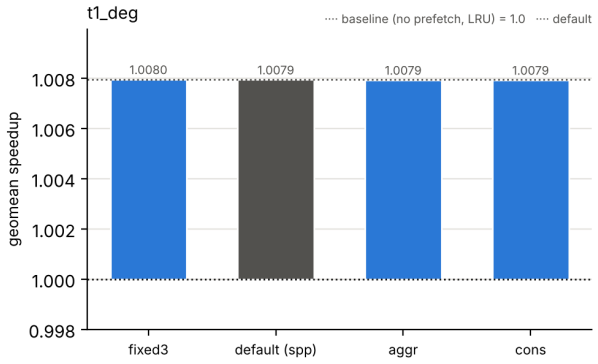
# Tuning experiments

Geometric-mean speedup over the baseline for each tuning variant (`results/tuning/`). Dotted lines: baseline (1.0) and the default.

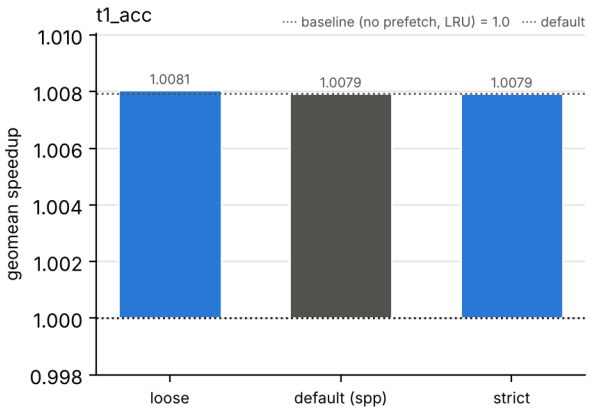
## A.1 Task 1B: SPP (L2C: LRU)



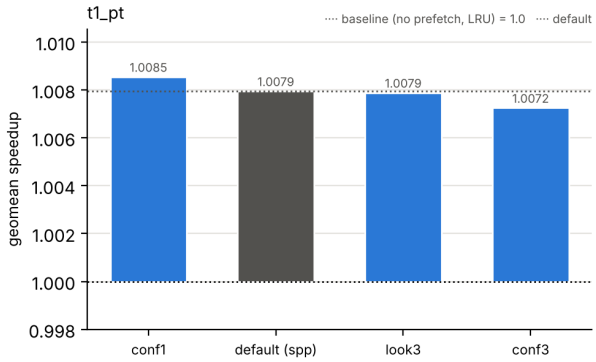

(a) Signature table size: 32 / 128 / 256



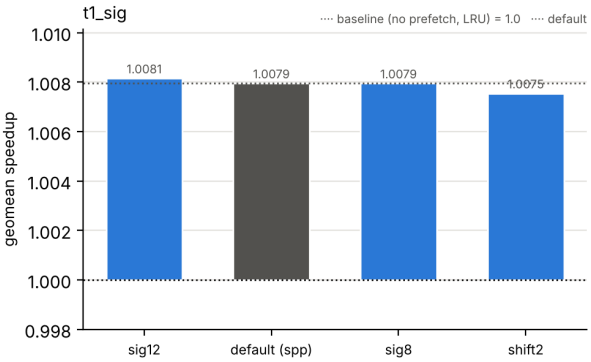

(b) Degree table: fixed 3 / 6-4-3-1-0 / 3-2-1-1-0



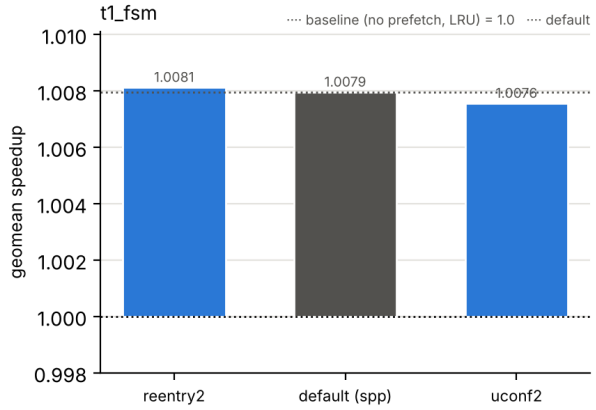

(a) Accuracy thresholds ±0.1



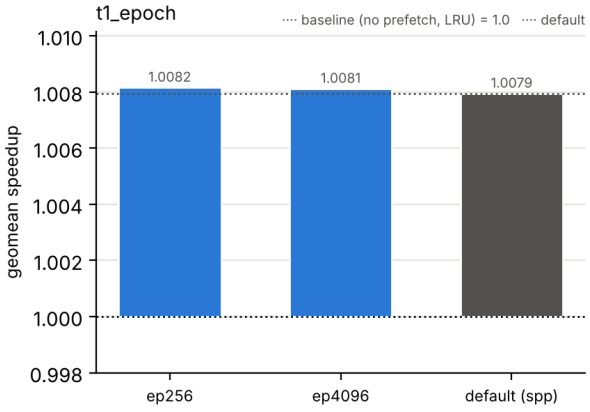

(b) PT confidence 1 / 3, lookahead confidence 3



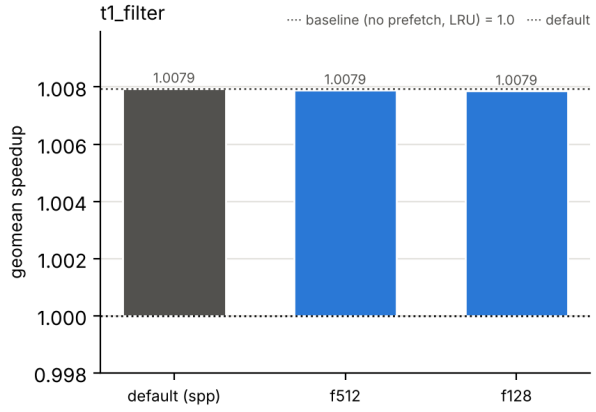

(a) Signature 8 / 12 bits, shift 2




(b) Uniform confidence 2, re-entry 2




(a) Throttling epoch 256 / 4096




(b) Prefetch filter 128 / 512

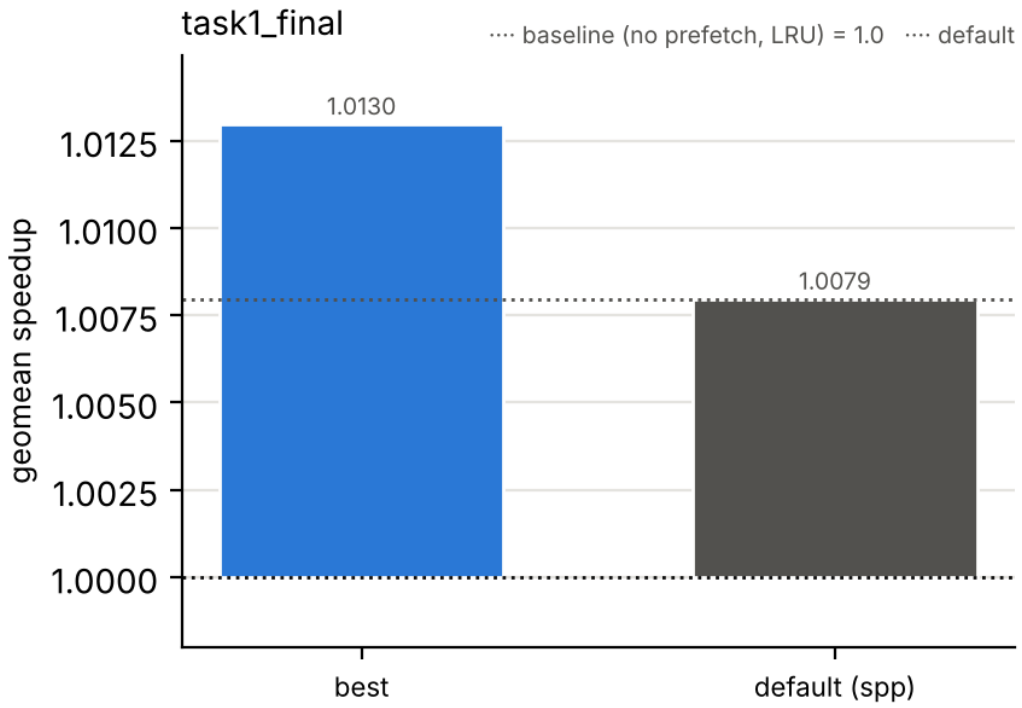
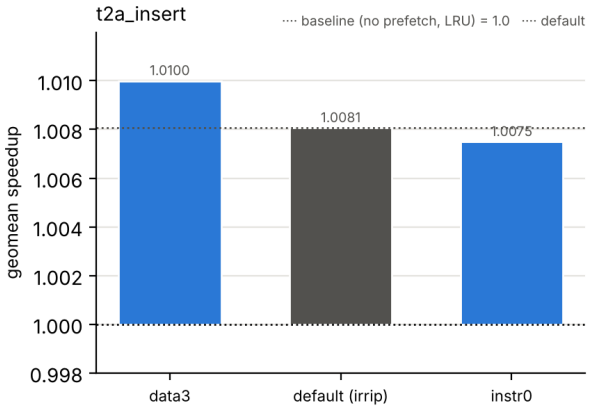
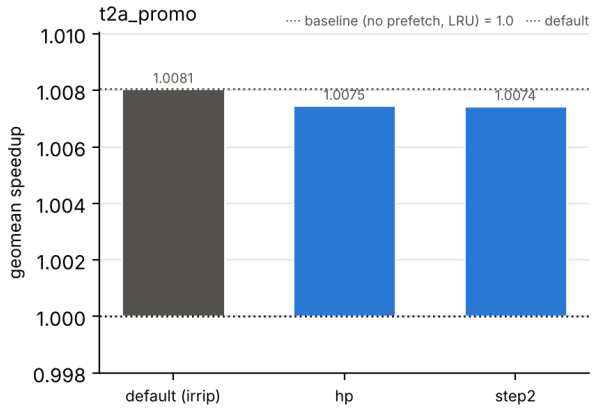



Figure A.5: Task 1B: combined winners vs. default.

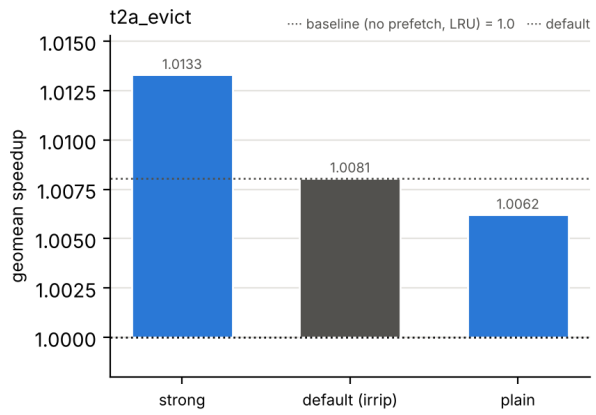
## A.2 Task 2A: I-RRIP (no prefetcher)



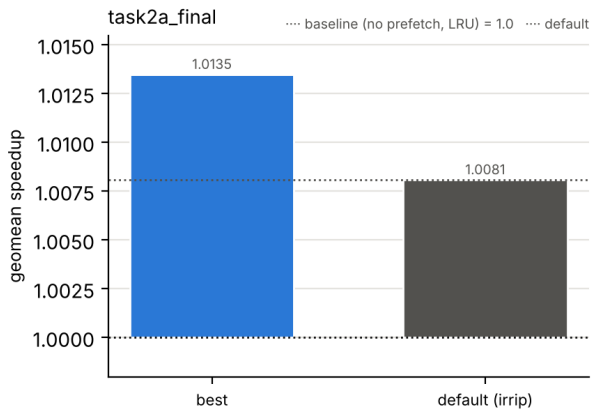

(a) Insertion: instructions at 0 / data at 3




(b) Promotion: data hit $\rightarrow$ 0 / $-2$

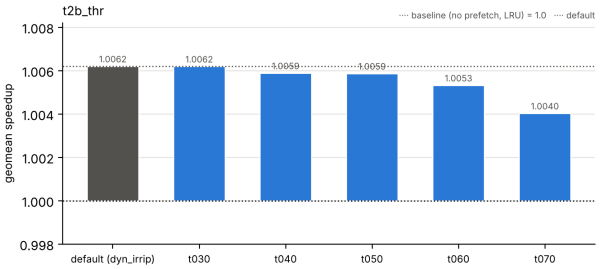



(a) Eviction: plain SRRIP / mode 2

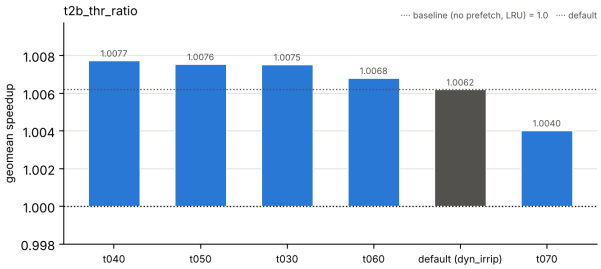



(b) Combined winners vs. default

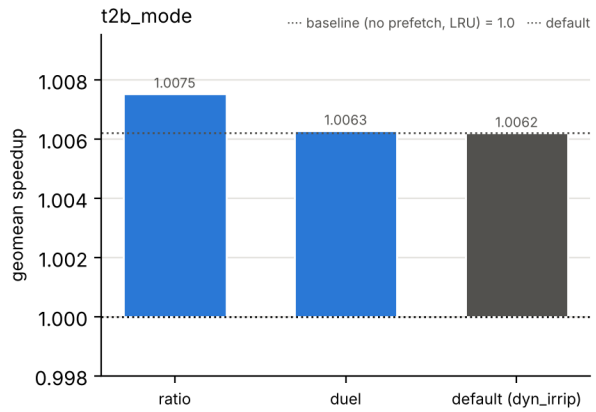
## A.3 Task 2B: Dynamic I-RRIP (no prefetcher)



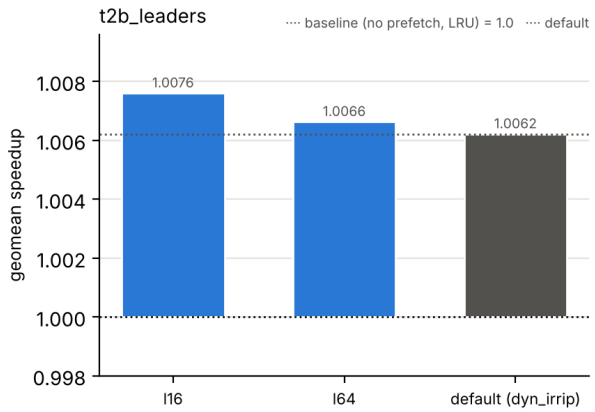

(a) Threshold $T$ 0.30–0.70, ratio AND guard



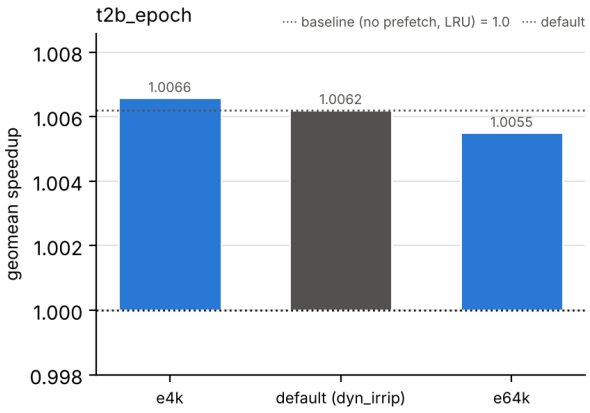

(b) Threshold $T$ 0.30–0.70, ratio only



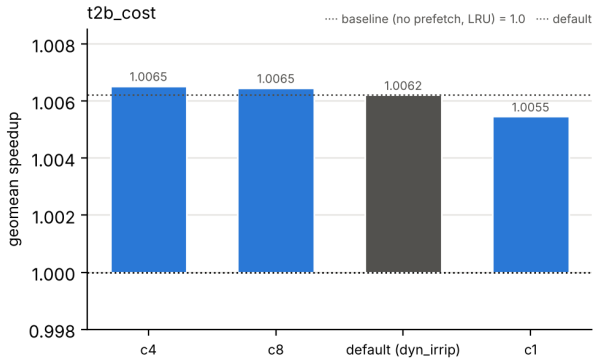

(a) Decision: ratio only / duel only



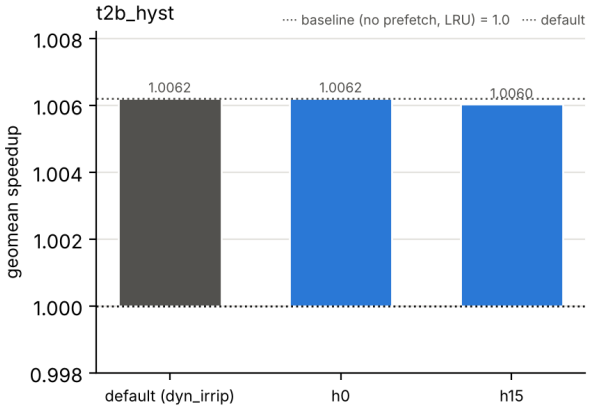

(b) Leader sets 16 / 64 per policy



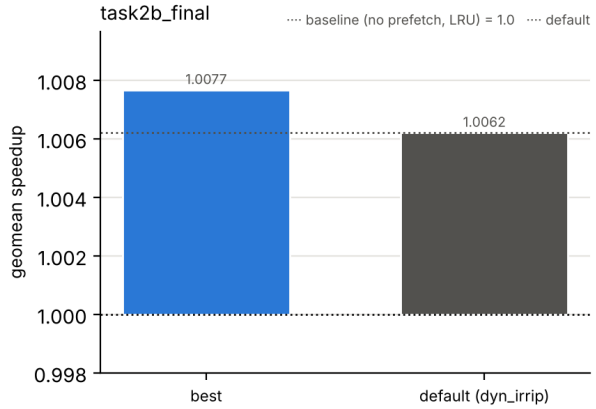

(a) Epoch 4K / 64K accesses




(b) Guard instruction-miss weight $c = 1$ / 4 / 8

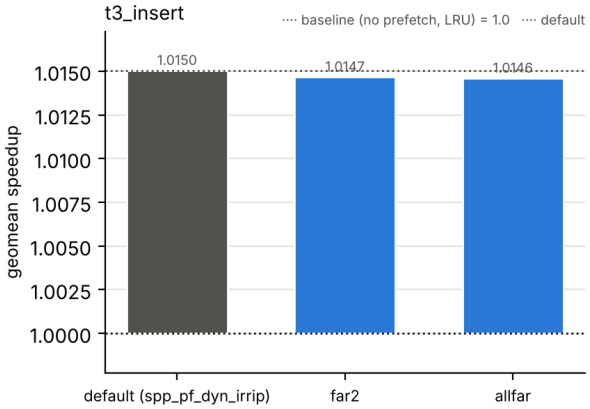



(a) Hysteresis: none / wide band

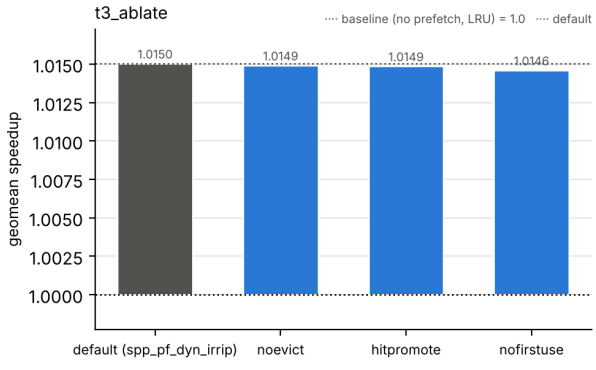



(b) Combined winners vs. default

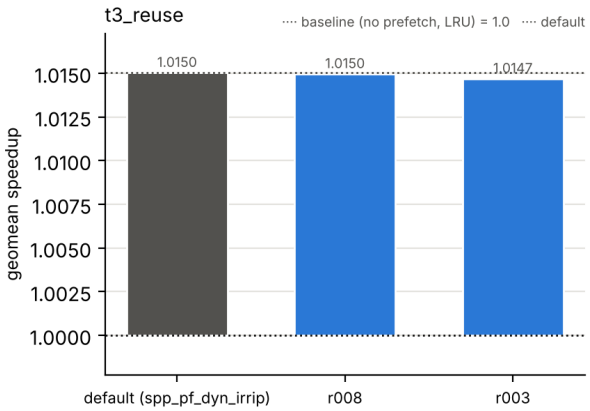
## A.4 Task 3: SPP + prefetch-aware dynamic I-RRIP



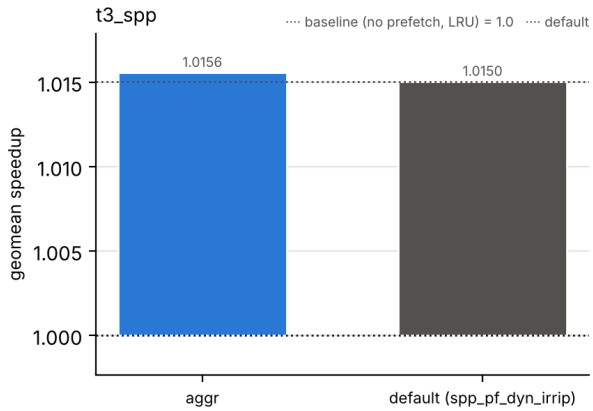

(a) Prefetch insertion: far = 2 / everything far




(b) Ablation: each mechanism off, hit promotion on




(a) Reuse threshold 0.03 / 0.08




(b) Aggressive SPP degrees